

DA2304 — Introduction to Computer Systems

Assignment 3: Jack Compiler Frontend & 2D Spatial Convolution

Rohan Sirsewad
Roll Number: DA24B023
Department of Data Science and AI
Indian Institute of Technology Madras

May 20, 2026

1. Executive Project Summary

This report documents the architectural design, algorithmic implementation, and validation of the Jack Compiler frontend alongside a 2D Spatial Convolution application written natively in the Jack language.

The deliverables are structured in two core components:

1. **High-Level Application Layer:** A spatial 2D convolution engine implementing matrix processing and edge sharpening algorithms within `Conv.jack` and driver execution via `Main.jack`.
2. **Compiler Infrastructure Layer:** A modular python-based parser suite consisting of a lexical tokenizer (`JackLexicalLexer`), code generator (`SyntaxAnalyzer`), scoped state manager (`LexicalScopeTable`), and stack-command emitter (`BytecodeWriter`), integrated with the virtual machine translator (`InstructionReader` and `AssemblyEmitter`).

2. Architectural Breakdown

The refactored compiler frontend follows a highly decoupled, modular object-oriented design using clear boundaries of single responsibility:

`JackTokenizer.py` (`JackLexicalLexer`) Ingests raw source text, strips nested and inline comment blocks through a deterministic character character lookup state manager, and emits categorized tokens (keywords, symbols, constants, and identifiers) into an structured XML format (`<Class>T.xml`).

`CompilationEngine.py` (`SyntaxAnalyzer`) A recursive-descent parser mapping token patterns to the formal grammar rules of the Jack language. In a highly optimized single-pass approach, it simultaneously verifies syntactical validity, writes the resulting structural parse tree XML (`<Class>.xml`), and translates active expressions into stack bytecode statements.

`SymbolTable.py` (`LexicalScopeTable`) Handles identifier mappings during compilation across class and subroutine scopes. Resolves variable attributes into a tuple containing type, scope kind (static, field, argument, local), and index offsets.

`VMWriter.py` (`BytecodeWriter`) An abstraction wrapper that structures formatting details for emitting stack-based VM instructions (`push`, `pop`, `call`, `function`, `return`) to

output files.

`JackCompiler.py` Orchestration driver that processes the source files sequentially, executing tokenization and parser compilation workflows.

The single-pass compilation architecture (parsing and code generation happening in tandem) minimizes allocation costs, bypassing the need to generate and traverse a large, intermediate Abstract Syntax Tree (AST) in memory.

3. Lexical Tokenizer Design

3.1 State-Machine Driven Comment Stripping

A common issue in tokenizer design is comment stripping. Utilizing standard single-line regular expressions (such as applying `/\*.?*/` line-by-line) fails on multi-line block comments because the dot operator does not consume newlines.

Consider a multi-line block comment:

```
1 /* Starting a multi-line comment
2    extending to this line */
3 let matrixSize = 5;
```

If processed line-by-line:

- Line 1 contains the open token `/*` without a closing token. A single-line regex fails to match and leaves the open block untouched.
- Line 2 contains only the close token `*/` with no opening block, leading to similar regex match failure and syntax compilation errors.

Even applying regex globally across the entire file can be fragile; a greedy match would consume valid, operational code placed between distinct block comments. To resolve this, our tokenizer implements a deterministic character character scan state-machine. By maintaining boolean flags (such as whether the cursor resides inside a string literal, line comment, or block comment), it guarantees robust, context-sensitive comment stripping without backtracking overheads.

3.2 Token Classification and XML Escaping

Once stripped, the code is parsed into lexical categories based on character types: symbols, numeric digits, strings bounded by double quotes, keywords, or identifiers. Special XML entities (`<`, `>`, `&`, `"`) are escaped safely into their matching XML representations (`<`, `>`, `&`, `"`). The replacement of the ampersand symbol (`&`) is always executed first to prevent secondary double-escaping bugs (which would translate `<` into `&lt;`).

4. Recursive-Descent Parser Design

The parser (`SyntaxAnalyzer`) validates the token stream against the formal Jack grammar using a recursive-descent strategy. Each non-terminal is parsed by its own dedicated method (e.g., `parseclass`,

4.0 Lookahead Navigation

The parser walks the token stream sequentially. The following primitives handle structural tracking:

- `_require_token(type, value)`: Asserts that the active token matches the expected constraints, appends its terminal node to the XML structure, and advances the token cursor.
- `_matches_token(type, value)`: Peeks at the active token to determine layout branch choices without advancing the cursor.
- `_lookahead_next_token()`: Looks ahead exactly one step past the active token, returning its properties.

4.1 The LL(2) Challenge in `parse_term`

While the Jack grammar is mostly LL(1), parsing expressions involving terms that begin with an identifier presents an LL(2) challenge. Upon encountering an identifier, the parser cannot distinguish between a simple variable (e.g., `x`), an array lookup (e.g., `arr[i]`), or a subroutine call (e.g., `foo()` or `obj.bar()`) using only one token of lookahead.

To resolve this, the parser peeks at the next token:

- If the next token is `[`, it parses it as an array indexing term.
- If the next token is `(` or `.`, it parses it as a subroutine call.
- Otherwise, it treats it as a standard variable identifier.

4.2 Expression Evaluation and Precedence

Jack implements strict left-to-right evaluation with no operator precedence. Expression parsing is structured inside the `parse_expression` method:

```

1 parse_expression:
2     parse_term()
3     while current_token in math_operators:
4         operator = require_token(symbol)
5         parse_term()
6         emit_bytecode(operator) // postfix arithmetic emit

```

Listing 1: Syntactic Parsing of Expressions

Non-primitive operations such as multiplication (`*`) and division (`/`) are translated into standard OS subroutine library calls: `Math.multiply` and `Math.divide`.

5. Lexical Scope Table Management

The compiler maintains a two-level scope architecture implemented in `LexicalScopeTable`:

- **Class-Level Scope**: Tracks static variables (mapped to the static segment) and instance fields (mapped to the `this` segment).
- **Subroutine-Level Scope**: Tracks method/function arguments (mapped to the argument segment) and local variables (mapped to the local segment). This table is completely cleared at the beginning of each subroutine declaration.

Identifier resolution always checks the subroutine scope first, falling back to class scope. If the identifier is absent in both, the parser processes it as a class name (e.g. for static calls like `Output.println`).

5.1 Symbol Table State inside Conv.conv2d()

At the exact step where the inner-loop accumulator is updated (`let cellVal = cellVal + ...`), the symbol tables hold the states described in Table 1 and Table 2:

Table 1: Class-Level Scopes for Conv

Variable Identifier	Data Type	Scope Kind	Segment Index
kernelMatrix	Array	static	0
strideStep	int	static	1
inputTensor	Array	field	0
outputTensor	Array	field	1
inputSize	int	field	2

Table 2: Subroutine-Level Scopes for conv2d

Variable Identifier	Data Type	Scope Kind	Segment Index
this	Conv	argument	0
dimOut	int	local	0
rOut	int	local	1
cOut	int	local	2
rIn	int	local	3
cIn	int	local	4
rF	int	local	5
cF	int	local	6
cellVal	int	local	7
outIdx	int	local	8

Thus, the variable `cellVal` resolves to index 7 of the local segment, and fields like `inputTensor` map to index 0 of the `this` segment. The implicit object pointer is passed as argument 0 and mapped to the pointer 0 segment at the method's entry.

6. VM Stack Generation Lowering

6.1 Constructor Memory Allocations

During class instantiation, the compiler determines the count of instance field variables. It emits a push constant `<fieldscount>` *instruction followed by* `acall toMemory.alloc 1t`

6.2 Method Preamble and Invocations

Methods operate on an active object instance passed implicitly at argument index 0. On entry, the method executes a preamble:

```

1 push argument 0
2 pop pointer 0

```

This maps all references in the `this` segment to the object's instance fields. When compiling a method call, the compiler pushes the object reference onto the stack as the first argument, followed by the explicit arguments, and increments the argument count by 1.

6.3 Safe Array Write Abstractions

Array lookups use the pointer 1 / that 0 translation. However, array writes (e.g. `let a[i] = expr;`) require care because evaluating the right-hand side expression might contain array accesses itself, which would clobber the pointer 1 register. To prevent this, the compiler pushes the base address plus the index offset onto the stack, compiles the right-hand side expression, pops the expression result to a temporary register (temp 0), updates pointer 1, and pop-assigns the value to that 0:

```
1 push arr          // push array base address
2 push index        // push index offset
3 add               // compute target address on stack
4 ... evaluate expression ...
5 pop temp 0        // save expression output
6 pop pointer 1     // update THAT register with target address
7 push temp 0       // restore expression output
8 pop that 0        // RAM[THAT] = expression output
```

Listing 2: Secure Array Write VM Bytecode

6.4 Compilation Trace for `let outputTensor[outIdx] = cellVal`

Using a 5×5 matrix test case where the first output pixel yields a value of 7 at flat index offset 0, the compiler generates the following trace:

```
1 push this 1       // push outputTensor base address (RAM 2050)
2 push local 8      // push outIdx value (0)
3 add               // RAM[SP-2] + RAM[SP-1] = 2050
4 push local 7      // push cellVal accumulator (7)
5 pop temp 0        // temp 0 = 7
6 pop pointer 1     // THAT = 2050 (pop target address to pointer 1)
7 push temp 0       // push 7
8 pop that 0        // RAM[2050] = 7
```

Listing 3: Trace for Array Assignment

The temporary register holds the output value while the destination address is safely loaded into the THAT base register, avoiding address clobbering.

7. Convolution Application Design

7.1 Algorithmic Choices

Stride Length (S) Set to 1 in the constructor. Convolving a 5×5 input with a 3×3 kernel yields a 3×3 output.

Filter Kernel A 3×3 Laplacian sharpening kernel $[0, -1, 0, -1, 5, -1, 0, -1, 0]$. This highlights high-frequency details by computing the difference between a pixel and its neighbors.

Flattening representation Matrices are flattened into 1D arrays in row-major format:

$$\text{Index}(r, c) = r \cdot \text{inputSize} + c$$

7.2 Output Boundary Calculations

The spatial convolution output side dimension (O) is calculated as:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

For $N = 5, F = 3, P = 0, S = 1$: $O = \lfloor 2/1 \rfloor + 1 = 3$. For $N = 9, F = 3, P = 0, S = 2$:
 $O = \lfloor 6/2 \rfloor + 1 = 4$.

7.3 Asymptotic VM Instruction Complexity

The 2D convolution algorithm runs four nested loops (two for the output grid dimensions, and two for the filter dimensions):

- Outer loops run O^2 times.
- Inner loops run $F^2 = 9$ times (a constant).

The total iterations scale as $9 \cdot O^2$. Since $O = \frac{N-3}{S} + 1$:

$$\text{For } S = 1: \text{ Iterations} = 9(N - 2)^2 = \Theta(N^2)$$

$$\text{For } S = 2: \text{ Iterations} = 9 \left(\frac{N - 1}{2} \right)^2 \approx \frac{9}{4} N^2 = \Theta(N^2)$$

Thus, changing the stride length from $S = 1$ to $S = 2$ decreases the absolute VM instruction count by approximately a factor of 4, but the asymptotic complexity in terms of the input size N remains **exactly identical** ($\Theta(N^2)$). Stride only influences the constant coefficient factor.

8. Verification and Test Summary

Validation was executed through systematic automated tests:

1. **Lexical Verification**: Verified that ConvT.xml correctly output all individual token elements and stripped multi-line block comments.
2. **Syntactic Parse Tree**: Verified correct element nesting in Conv.xml.
3. **VM Bytecode Generation**: Verified correct method preambles, constructor allocations, and array pop operations.
4. **Numerical Veracity**: Confirmed that the output printed under the CPU emulator matches the expected Laplacian convolution matrix for a 5×5 matrix:

```
Conv Output (5x5, S=1):
7 8 9
12 13 14
17 18 19
```

9. Bugs and Troubleshooting Observations

9.1 The Method Argument Offset Bug

During early integration, method calls generated corrupted variables. In Jack, method calls are compiled by pushing the parent object as argument 0, followed

by the explicit arguments. A call like `convolutionEngine.initFilter(filterKernel)` must be compiled as `call Conv.initFilter 2`.

Our initial implementation did not increment the argument count for method calls. The callee method read local stack values instead of the object reference, causing the method preamble to overwrite the parent's pointer segment. We resolved this by querying `self._scope_table.has_symbol(initial_identifier)in the parser to distinguish method calls from stati`

9.2 Handling Double XML Escapes

We encountered a bug where character escaping resulted in double-escaped sequences. Iterating over the replacement mapping in an arbitrary dictionary order replaced the ampersand character `&` in already-escaped entities (e.g. converting `< first` to `<`, then to `&lt;`). We resolved this by ensuring the ampersand character is replaced first, before any other entities.

10. Conclusion

This project demonstrated the translation of high-level object-oriented Jack code into VM instructions and native assembly. Developing the symbol table and parser highlighted the mechanics of stack frames, method preambles, and the lowering of complex constructs down to basic Hack CPU operations. The spatial convolution application provided a thorough test of all language features, validating the compiler frontend.